Flask + HTML Day — Student Instructions

Wednesday | Build a Web GUI to Control Your LED Strip

The Big Picture

Today you are building a real web application. By the end of the session, you will be able to open a browser on any device connected to the classroom Wi-Fi, click a button, and watch the LED strip change color in real time.

Your Browser ---> HTTP ---> **Flask (Pi)** ---> MQTT ---> **ESP32** ---> wire ---> **LED Strip**

Step 1 — Install Flask and paho-mqtt on the Raspberry Pi

On the Raspberry Pi (SSH terminal), run:

```
pip install flask paho-mqtt
```

This only needs to be done once. If it says already installed, move on.

Step 2 — Set Up Your Project Folder

Create a folder on the Pi to hold your project files. Everything must stay together or Flask won't find the HTML page.

```
mkdir ~/led_controller
cd ~/led_controller
mkdir templates
```

Copy app.py into ~/led_controller/ and index.html into ~/led_controller/templates/

Your folder should look like this:

```
led_controller/
    app.py
    templates/
        index.html
```

Step 3 — Start the Flask Server

Make sure you are in the led_controller folder, then run:

```
cd ~/led_controller
python3 app.py
```

You should see output like:

```
LED Controller Web Server
Open your browser and go to:
  http://localhost:5000
  http://192.168.x.x:5000
Press Ctrl+C to stop the server.
```

Leave this terminal open and running while you work. Open a second SSH terminal if you need to run other commands.

Step 4 — Open the Page in Your Browser

On any device on the same Wi-Fi network, open a browser and type the Pi's IP address with port 5000:

```
http://192.168.x.x:5000   (replace x.x with your Pi's actual IP)
```

Find your Pi's IP by running: hostname -I (the first number is it)

You should see the LED Controller page. Click a color button and watch the LED strip!

Step 5 — Customize the Web Page (index.html)

Open templates/index.html in a text editor on the Pi (or edit it on your computer and copy it over). The file has clear comments guiding you to the right spots.

Task	Where to look in index.html	Difficulty
Change the page title and subtitle	HEADER section (near top of body)	Easy
Change a button label	Button text inside the <button> tags	Easy

Change a button swatch color	CSS: #btn-red .color-swatch { background-color: }	Easy
Add a new color button	BUTTON GRID + CSS + app.py + main.py	Medium
Change page background / card color	CSS :root variables at the top of <style>	**Easy**
Enable the RGB color picker	CSS .rgb-section: change display:none to display:block	Medium
Add your name in the footer	FOOTER section at the bottom of body	Easy

Step 6 — How to Add a New Color (walkthrough)

Adding a color requires changes in three files. Here is the full walkthrough for orange:

1. index.html — add the button in the BUTTON GRID section:

```
<button class="color-btn" id="btn-orange" data-color="orange">
  <div class="color-swatch"></div>
  Orange
</button>
```

2. index.html — add the swatch color in the CSS section:

```
#btn-orange .color-swatch { background-color: orange; }
```

3. app.py — add orange to ALLOWED_COLORS:

```
ALLOWED_COLORS = {"red", "green", "blue", "white", "off", "orange"}
```

4. main.py on the ESP32 — add an elif inside on_msg():

```
elif command == "orange":
    fill(255, 100, 0)   # mix of red + a little green = orange
```

After editing main.py, save it to the ESP32 in Thonny and press reset. Then restart app.py on the Pi. Now the orange button will work end to end.

Stretch Goals

Enable the RGB color picker

In index.html, find .rgb-section in CSS and change display:none to display:block. Then add RGB handling to main.py: parse messages like 'rgb:255,100,0' and call fill().

Add a brightness slider

Add a range input in HTML. Send the value to a new Flask route /brightness/. Have app.py publish 'bright:40' to MQTT, and parse it in main.py to change BRIGHTNESS.

Make buttons animate the LEDs (e.g. flash)

Send rapid on/off messages using JavaScript setInterval() and a Flask route for 'flash'.

Style the page to match a theme

Change all the CSS :root variables to a new color scheme. Give it a name and a logo.

Troubleshooting

- **"Connection refused" in browser**: Flask is not running. Check the Pi terminal — look for error messages, then restart app.py.

- **Page loads but buttons do nothing**: Check the Pi terminal for errors. Confirm Mosquitto is running (sudo systemctl status mosquitto). Make sure the ESP32 main.py is running (LEDs should be dim blue).

- **"Internal Server Error"**: Look at the Flask terminal — it will show a Python traceback. Read the last line to find the error.

- **New color button does not work**: Check all four steps in Step 6. A common miss: forgetting to add it to ALLOWED_COLORS in app.py, or forgetting to handle it in main.py on the ESP32.

- **Cannot reach Pi from browser**: Confirm the device and Pi are on the same Wi-Fi. Try pinging: ping 192.168.x.x from the device. If no response, ask your instructor — AP isolation may be on.

When your page works end-to-end, show your instructor before moving to stretch goals.